dYdX v4 Protocol 데이터 모델 분석

개요

dYdX v4 Protocol은 Go 언어와 Protobuf를 기반으로 구축된 데이터 모델을 사용합니다. 이 문서는 Protocol Layer의 핵심 데이터 구조체들과 그들의 직렬화 방식, 상태 저장 메커니즘을 상세히 분석합니다.

핵심 데이터 모델

1. Order (주문) 데이터 모델

Protobuf 정의

```
message Order {
OrderId order_id = 1;
Side side = 2;
uint64 quantums = 3;
uint64 subticks = 4;
oneof good_til_oneof {
uint32 good_til_block = 5;
fixed32 good_til_block_time = 6;
}
TimeInForce time_in_force = 7;
```

```
bool  reduce_only = 8;

uint32  client_metadata = 9;

ConditionType  condition_type = 10;

uint64  conditional_order_trigger_subticks = 11;

}
```

Go 구조체 분석

Order 구조체는 다음과 같은 핵심 필드들을 포함합니다:

- **OrderId**: 주문의 고유 식별자
- SubaccountId: 주문을 생성한 서브계정 ID
- ClientId: 클라이언트별 고유 ID
- OrderFlags: 주문 타입 플래그 (Short-Term, Long-Term, Conditional)
- ClobPairId: 거래 쌍 ID
- **Side**: 주문 방향 (BUY/SELL)
- **Quantums**: 주문 수량 (기본 단위)
- **Subticks**: 가격 레벨 (서브틱 단위)
- **TimeInForce**: 주문 유효 기간 정책
- UNSPECIFIED: 기본 동작 (매칭 후 메이커 주문으로 등록)
- IOC: Immediate or Cancel
- POST_ONLY: 메이커 주문만 허용
- FILL_OR_KILL: 전량 체결 또는 취소

주문 타입 분류

```
// OrderFlags를 통한 주문 타입 구분
const (
ShortTermOrder = 0x00  // 단기 주문
LongTermOrder = 0x40  // 장기 주문 (비트 1)
ConditionalOrder = 0x20  // 조건부 주문 (비트 2)
)
```

2. OrderId (주문 식별자) 데이터 모델

Protobuf 정의

```
message  OrderId {
dydxprotocol.subaccounts.SubaccountId  subaccount_id = 1;
fixed32  client_id = 2;
uint32  order_flags = 3;
uint32  clob_pair_id = 4;
}
```

고유성 보장 메커니즘

OrderId는 다음 필드들의 조합으로 전역 고유성을 보장합니다:

- **SubaccountId**: 계정별 격리
- **ClientId**: 계정 내 주문 구분
- **OrderFlags**: 주문 타입별 네임스페이스 분리

- **ClobPairId**: 거래 쌍별 분리

3. Subaccount (서브계정) 데이터 모델

Protobuf 정의

```
message SubaccountId {
string owner = 1;
uint32 number = 2;
}

message Subaccount {
SubaccountId id = 1;
repeated AssetPosition asset_positions = 2;
repeated PerpetualPosition perpetual_positions = 3;
bool margin_enabled = 4;
}
```

Go 구조체 분석

- **SubaccountId**: 서브계정 식별자
- Owner: 지갑 주소 (Cosmos 주소 형식)
- Number: 계정 번호 (최대 128,000개 지원)
- **AssetPosition**: 자산 포지션 배열
- 항상 asset_id 기준 오름차순 정렬
- 각 자산별 잔고 및 상태 정보

- **PerpetualPosition**: 영구 선물 포지션 배열
- 항상 perpetual_id 기준 오름차순 정렬
- 포지션 크기, 진입 가격, 미실현 손익 등

4. Fill (체결) 데이터 모델

체결 데이터는 주문 매칭 결과를 나타내며, Protocol에서 직접적인 구조체보다는 이벤트 형태로 처리됩니다.

체결 이벤트 구조

```
type Fill struct {
FillAmount uint64
MakerOrderId OrderId
TakerOrderId OrderId
MakerFee int64
TakerFee int64
Price uint64
}
```

Protobuf 직렬화 방식

1. 직렬화 최적화

dYdX v4는 성능 최적화를 위해 다음과 같은 Protobuf 설정을 사용합니다:

```
option go_package = "github.com/dydxprotocol/v4-chain/protocol/x/clob/types";
```

```
// 포인터 대신 값 타입 사용으로 메모리 효율성 향상
OrderId order_id = 1 [ (gogoproto.nullable) = false ];
```

2. 바이너리 인코딩

- **Varint 인코딩**: 정수 필드들은 가변 길이 인코딩 사용
- **Fixed32/Fixed64**: 고정 길이가 필요한 필드 (타임스탬프 등)
- **String 인코딩**: UTF-8 인코딩된 문자열

3. 버전 호환성

Protobuf의 하위 호환성을 활용하여 스키마 진화를 지원:

- 새 필드 추가 시 기본값 사용
- 필드 번호 재사용 금지
- Optional 필드를 통한 점진적 마이그레이션

상태 저장 및 조회 메커니즘

1. Cosmos SDK State Store

KVStore 기반 저장

```
// 주문 상태 저장 키 패턴
orderKey := []byte("order/" + orderID.String())
store.Set(orderKey, orderBytes)
```

```
// 서브계정 상태 저장
subaccountKey := []byte("subaccount/" + subaccountID.String())
store.Set(subaccountKey, subaccountBytes)
```

인덱싱 전략

- **Primary Key**: 엔티티별 고유 식별자
- **Secondary Index**: 쿼리 최적화를 위한 보조 인덱스
- 서브계정별 주문 목록
- 거래 쌍별 주문서
- 만료 시간별 주문 정렬

2. 메모리 기반 상태 관리

MemCLOB (인메모리 주문서)

```
type MemClob struct {
// 주문서별 매칭 엔진
orderbooks map[ClobPairId]*Orderbook
// 주문 ID별 빠른 조회
orders map[OrderId]*Order
// 서브계정별 주문 목록
subaccountOrders map[SubaccountId][]OrderId
}
```

상태 동기화

- **BeginBlocker**: 블록 시작 시 메모리 상태 초기화
- **EndBlocker**: 블록 종료 시 만료된 주문 정리
- **PrepareCheckState**: 다음 블록을 위한 상태 준비

3. 상태 조회 패턴

단일 엔티티 조회

```
func (k Keeper) GetOrder(ctx sdk.Context, orderId OrderId) (Order, bool) {
store := ctx.KVStore(k.storeKey)
key := types.OrderKey(orderId)
bz := store.Get(key)
if bz == nil {
return Order{}, false
}
var order Order
k.cdc.MustUnmarshal(bz, &order)
return order, true
}
```

범위 조회 (Iterator 패턴)

```
func (k Keeper) GetAllOrdersForSubaccount(ctx sdk.Context, subaccountId Subacc
store := ctx.KVStore(k.storeKey)
prefix := types.SubaccountOrderPrefix(subaccountId)
```

```
iterator := sdk.KVStorePrefixIterator(store, prefix)

defer iterator.Close()

var orders []Order

for ; iterator.Valid(); iterator.Next() {

var order Order

k.cdc.MustUnmarshal(iterator.Value(), &order)

orders = append(orders, order)

}

return orders

}
```

데이터 무결성 보장

1. 트랜잭션 원자성

- **Cosmos SDK Transaction**: 모든 상태 변경은 트랜잭션 내에서 원자적으로 처리
- **Rollback 메커니즘**: 오류 발생 시 전체 상태 변경 롤백
- **Multi-Store**: 모듈별 독립적인 상태 저장소

2. 검증 로직

주문 검증

```
func (o Order) Validate() error {

// 필수 필드 검증
```

```
if o.OrderId.SubaccountId.Owner == "" {
return errors.New("owner cannot be empty")
}
// 수량 검증
if o.Quantums == 0 {
return errors.New("quantums must be positive")
}
// 가격 검증
if o.Subticks == 0 {
return errors.New("subticks must be positive")
}
return nil
}
```

상태 일관성 검증

- **잔고 검증**: 주문 생성 시 충분한 잔고 확인
- **포지션 검증**: 포지션 크기와 마진 요구사항 검증
- **주문서 일관성**: 매칭 후 주문서 상태 검증

3. 이벤트 기반 상태 동기화

상태 변경 이벤트

```
// 주문 생성 이벤트
ctx.EventManager().EmitEvent(
```

```
sdk.NewEvent(
types.EventTypeOrderPlacement,
sdk.NewAttribute(types.AttributeKeyOrderId, orderId.String()),
sdk.NewAttribute(types.AttributeKeySubaccountId, subaccountId.String()),
),
)
```

Indexer와의 동기화

- **Kafka 이벤트**: 상태 변경 시 Kafka로 이벤트 전송
- **이벤트 순서 보장**: 블록 높이와 트랜잭션 인덱스 기반 순서 보장
- **재처리 메커니즘**: 이벤트 처리 실패 시 재시도 로직

성능 최적화

1. 메모리 사용 최적화

- **Object Pooling**: 자주 사용되는 객체의 재사용
- **Zero-Copy**: 불필요한 메모리 복사 최소화
- **Compact Encoding**: 효율적인 바이너리 인코딩

2. 조회 성능 최적화

- **캐싱 전략**: 자주 조회되는 데이터의 메모리 캐싱
- **인덱스 최적화**: 쿼리 패턴에 맞는 인덱스 설계
- **배치 처리**: 여러 엔티티의 일괄 조회

3. 저장 성능 최적화

- **배치 쓰기**: 여러 상태 변경의 일괄 처리
- **압축**: 저장 공간 절약을 위한 데이터 압축
- **비동기 처리**: 블로킹되지 않는 상태 저장

이러한 데이터 모델 설계를 통해 dYdX v4 Protocol은 높은 성능과 확장성을 제공하면서도 데이터 무결성과 일관성을 보장합니다.